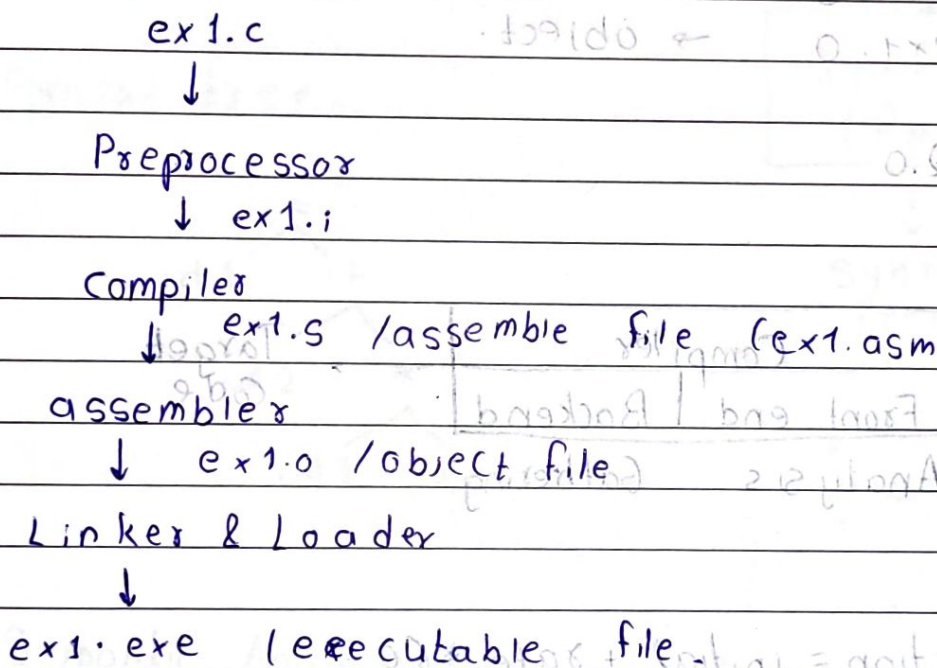


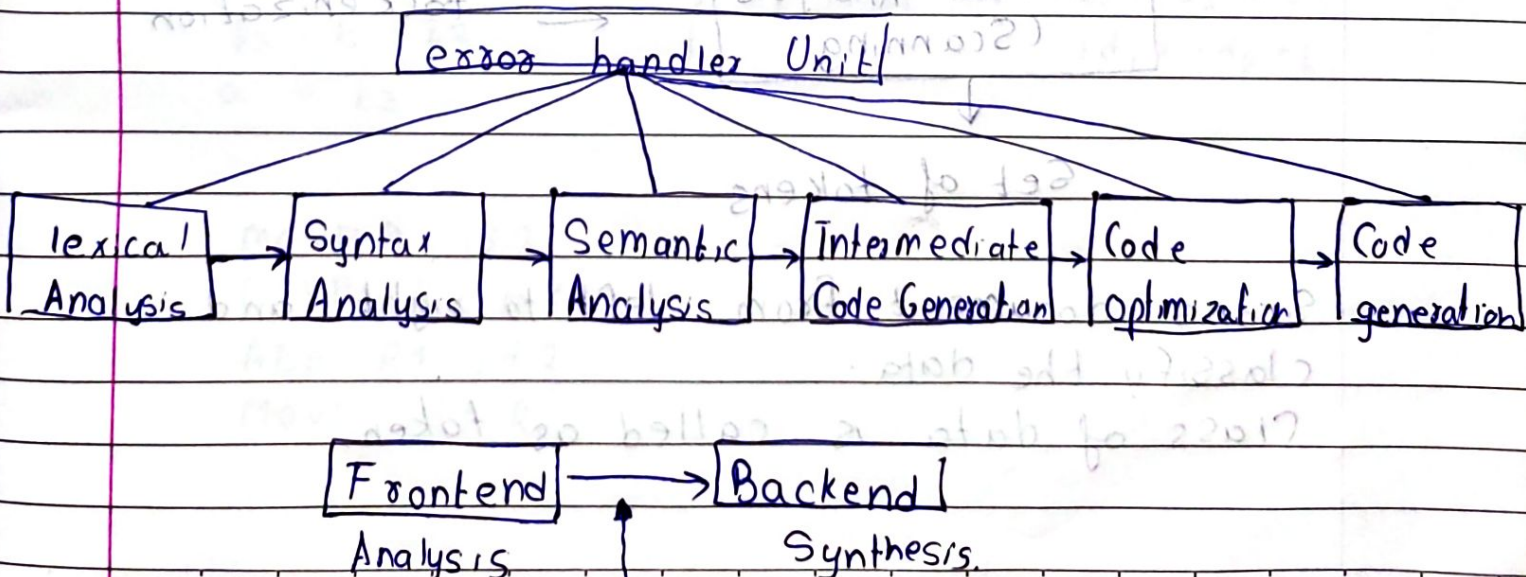
* Difference between compiler and interpreter (focused on functionalities and object orientation) & their Complexities

Compiler, interpreter, loader, linker, assembler, preprocessor definitions.

* Input language processing System.



* Types of Compiler : wide, narrow, big, small, cross, incremental.



Front-end structure method

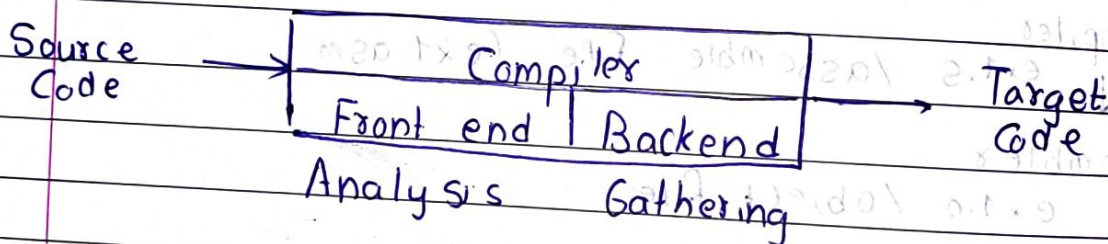
* Commands :

- wall
- Save temp (Saves all intermediate files.)
- verbose

* View -

- vi ex1.i → View Output File of preprocessor.
- vi ex1.s → assembly
- vi ex1.o → object.

* Collect 2.0



ex. Position = initial + rate * 60;
or a = b + c * 60;

Lexical Analysis
(Scanning)

tokenization

Set of tokens

Scans document from left to right and classify the data.
Class of data is called as token.

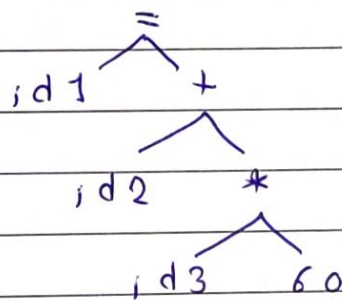
ex $a = b + c * 60;$

$\langle id1 \rangle \langle = \rangle \langle id2 \rangle \langle + \rangle \langle id3 \rangle \langle * \rangle \langle 60 \rangle \langle ; \rangle$

Symbol Table

1	id	a
2	id	b
3	id	c

Syntax tree.

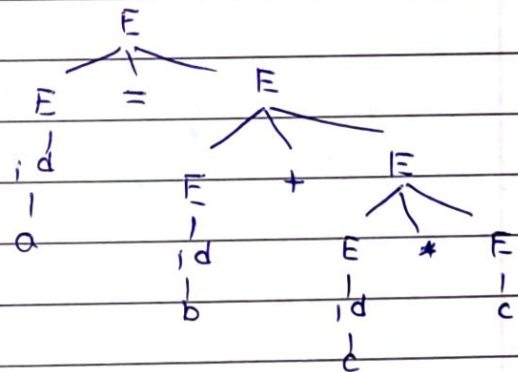


Set of tokens.

Syntax Analysis
(Parsing).

using CFG.

Syntax
tree



* Semantic Analysis
Three Address code.

$t_1 = \text{into real}(60)$

$t_2 = c * t_1$

$t_3 = b + t_2$

$a = t_3$

Optimize $\rightarrow$ $t_1 = id3 * 60$
 $id_1 = id_2 + t_1$

MOV R, id3

MOV R, #60

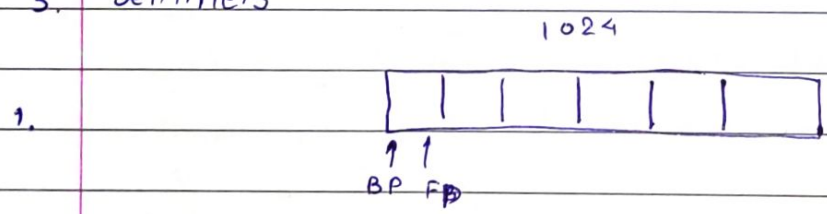
ADD R1, id2

MOV id1, R,

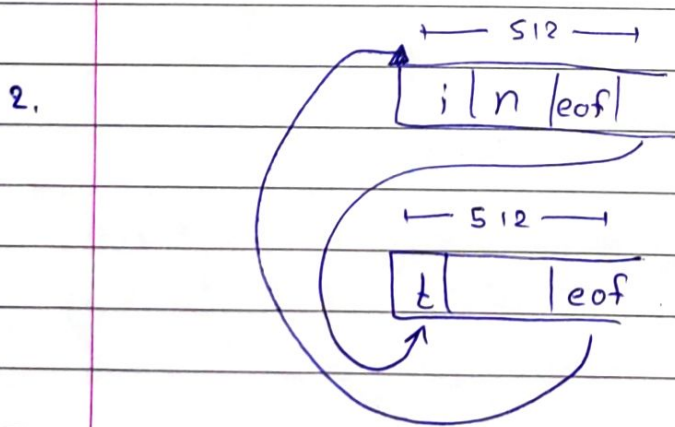
l

Input buffering.

1. Single buffer technique
2. Buffer Pair
3. Sentinels



Beginning ptr
 forwarding ptr



Input Source code is stored inside buffer.

Sentinels are special characters specially used to show end of file

Error detection

Error reporting

Error recovery

T_0 - Unrestricted G $\alpha \notin \epsilon$

T_1 - Context Sensitive G $| \alpha | \leq | \beta |$

T_2 $\rightarrow$ Context free G $| \alpha | = | \beta |$ & $\alpha \in V$

$T_3 \rightarrow$ Regular/Linear G

Identify the type of grammar

1. $AB \rightarrow BA$
 $B \rightarrow b$

Ans. T_1

2. $S \rightarrow aS / bS / \epsilon$

Ans. T_3

3. $S \rightarrow aSb / AB$

$A \rightarrow a$

$B \rightarrow b$

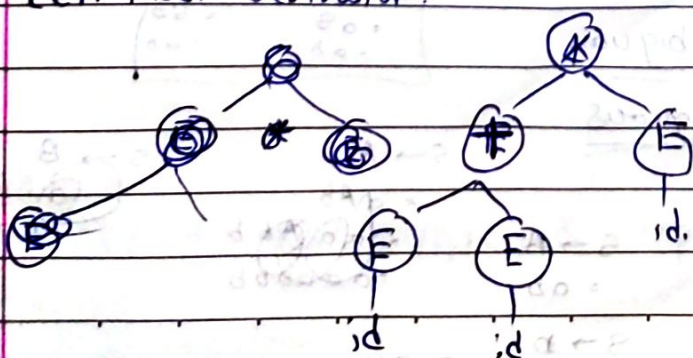
} T_2

Consider the given grammar

$E \rightarrow E + E \mid E * E \mid id$

derive the string $id + id * id$ by using left most & right most

Left most derivation.



$E \rightarrow E + E$

$\rightarrow id + E$

$\rightarrow id + E * E$

$\rightarrow id + id * id$

Left most

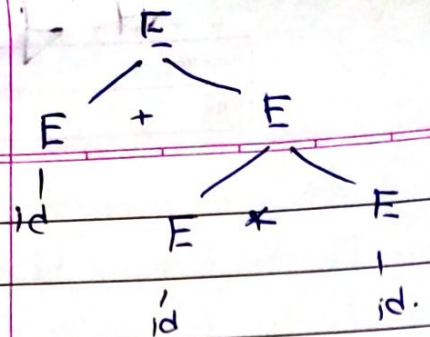
$E \rightarrow E * E$

$\rightarrow E * id$

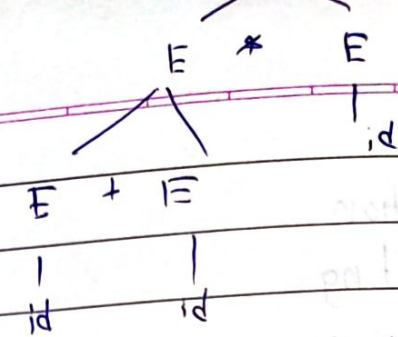
$\rightarrow E + E * id$

$\rightarrow id + id * id$

Right most

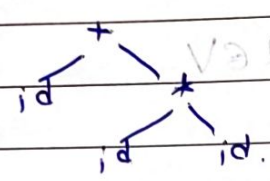


Left Most.

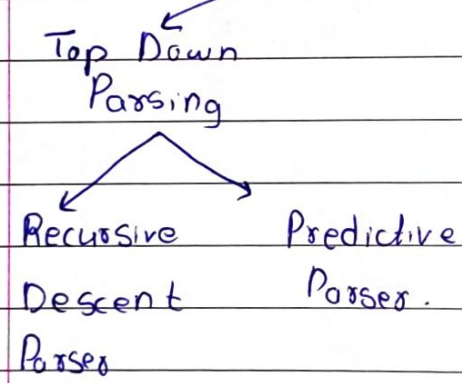


Right Most derivation.

Abstract Syntax Tree.



Passing



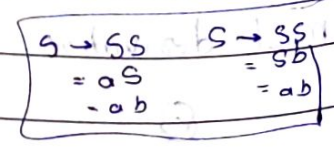
Bottom up Passing

- 1) $LL(1) / LL(k)$
- 2) Simplified Grammar.
- 3) Left-Right Scanning
- 4) Leftmost derivation
- 5) $k = \text{no of look aheads}$

- 1) Elimination of Ambiguity
- 2) Elimination of left recursion
- 3) Left factorization
- 4) Elimination of ϵ production
- 5) Elimination of Unit production
- 6) Elimination of useless production

Check whether the given grammar is ambiguous or not.

- 1) $S \rightarrow SSa|b \rightarrow \text{Ambiguous}$
- 2) $S \rightarrow AB \rightarrow \text{Ambiguous}$
- 3) $A \rightarrow aAb|ab$
- 4) $B \rightarrow abB|G$

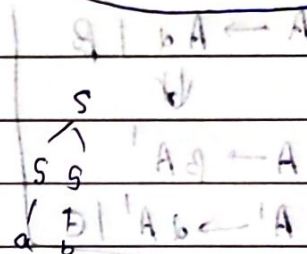
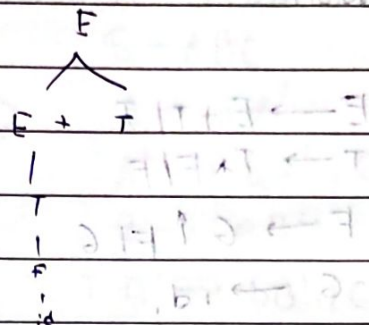
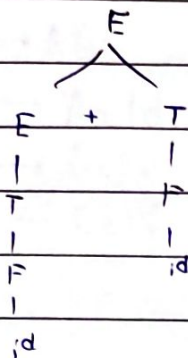


$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$

$E \rightarrow E + T$
 $= T + T$
 $= T * F + T$
 $= F * F * F$
 $= id * id + id$

$E \rightarrow E + T$
 $= E + F$
 $= E + id$
 $= T + id$
 $= F + id$
 $= id + id$

M T W T F S S
 Page No.: $E = T + T$
 YOUVA
 Date: $= F + T$
 $= id + T$
 $= id + F$
 $= id + id$



When the operators are left associative remove right recursion and if the operator is right associative remove left recursion.

$T = \{ +, *, \uparrow, id \}$
 $id > \uparrow > * > +$

* Levelize the given grammar according to the priority

$id > \uparrow > * > +$
 $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow E \uparrow E$
 $E \rightarrow id$

Use new variables while removing left recursion

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow G \uparrow F$
 $G \rightarrow id$

• LL(1) grammar is input to top-down parser

* Elimination of left recursion:

$$A \rightarrow A d \mid B$$

↓

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow G \mid F \mid G$$

$$G \rightarrow id.$$

$$A \rightarrow A d_1 \mid A d_2 \mid B_1 \mid B_2 \dots B_n$$

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \dots$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \dots \mid \epsilon$$

$$E' \rightarrow T E'$$

$$E' \rightarrow \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$F \rightarrow G \mid F \mid G$$

$$G \rightarrow id.$$

Remove left recursion from given set of gr

$$A \rightarrow A B d \mid A a \mid a.$$

$$B \rightarrow B e \mid b$$

A

$$A \rightarrow a A' \mid \epsilon$$

$$A' \rightarrow B d A' \mid \epsilon \mid a A' \mid \epsilon$$

$$B \rightarrow b B'$$

$$B' \rightarrow e B' \mid \epsilon$$

* Indirect Recursion:

$$X \rightarrow X S b \mid S a \mid b$$

$$S \rightarrow S b \mid X a \mid b.$$

→

$$X \rightarrow X S b \mid X a d \mid b$$

$$S \rightarrow S b \mid S a a b.$$

$$X \rightarrow b X'$$

$$X' \rightarrow S b X' \mid a a X' \mid \epsilon$$

$$S \rightarrow b S'$$

$$S' \rightarrow b S' \mid a a S' \mid \epsilon$$

XY | YX | XX | YY | XYX | YXX

Q Remove Left recursion from given grammar

$$S \rightarrow ABC$$

$$A \rightarrow Aa|d \Rightarrow A \rightarrow dA'$$

$$B \rightarrow Bb|e \Rightarrow A' \rightarrow aA'|E$$

$$C \rightarrow Cc|f \Rightarrow B \rightarrow eB'$$

$$B' \rightarrow bB'|E$$

$$C' \rightarrow cC'|E$$

$$C'' \rightarrow cC''|E$$

Left Factorization

If the production is of the form $A \rightarrow a_1 A' | a_2 A' | a_3 A' | \dots$ then left factoring of the production is

$$A \rightarrow a A' : A' \rightarrow a_1 A'' | a_2 A'' | a_3 A'' | \dots$$

Q Left factor the given grammar.

1. $A \rightarrow aAB | aAC | aBC$

$$B \rightarrow b$$

$$C \rightarrow c$$

$$A \rightarrow aA'$$

$$\Rightarrow A' \rightarrow AB | AC | BC$$

$$B \rightarrow b$$

$$C \rightarrow c$$

$$A \rightarrow aA'$$

$$A' \rightarrow AA'' | BC$$

$$A'' \rightarrow B | C$$

$$B \rightarrow b$$

$$C \rightarrow c$$

2. $S \rightarrow E \mid S \mid E \mid S \mid E \mid S \mid a$

$$E \rightarrow b$$

$$S \rightarrow E \mid S \mid S' \mid a$$

$$S' \rightarrow e \mid S$$

$$E \rightarrow b$$

3. $S \rightarrow a \mid ab \mid abc \mid abcd$

$$\Rightarrow S \rightarrow aS'$$

$$S' \rightarrow b \mid bc \mid bcd \mid e$$

$$\Rightarrow S \rightarrow aS'$$

$$S' \rightarrow bS'' \mid e$$

$$S'' \rightarrow c \mid cd \mid e$$

$$\Rightarrow S \rightarrow aS'$$

$$S' \rightarrow bS'' \mid e$$

$$S'' \rightarrow cS''' \mid e$$

$$S''' \rightarrow d \mid e$$

4) Elimination of ϵ -transition / Null transition production

If there exists production

$$S \rightarrow XYX$$

$$X \rightarrow \epsilon$$

$$Y \rightarrow \epsilon$$

$$S \rightarrow XYX \mid Y \mid XX \mid XY \mid YX \mid X$$

$$X \rightarrow \epsilon$$

$$Y \rightarrow \epsilon$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow id$$

$$E \rightarrow TE' \mid T$$

$$E' \rightarrow +TE' \mid +T$$

$$T \rightarrow FT' \mid F$$

$$T' \rightarrow *FT' \mid *F$$

$$F \rightarrow id$$

5) Elimination of Unit Production.

6) Removal of Useless Symbols.

$$S \rightarrow AB$$

$$A \rightarrow a \mid b$$

$$B \rightarrow c \mid d$$

$$C \rightarrow C \mid F$$

Useless.

After Simplification we can use bottom up
 $\hookrightarrow$ LL(1) Grammar

Top Down.

Recursive

Descent
Parser

Predictive / LL(1) /

Table driven parser.

$$E \rightarrow num \mid T$$

$$T \rightarrow *num \mid E$$

E()

T()

```
{ if lookahead = num
  { match(num);
    T(); }
else {
  error();
  if lookahead = $
    print("Success");
  else
    error();
}
```

```
{ if lookahead = * then
  match(*);
  if lookahead = num
    match(num);
    T();
  else
    error();
  else lookahead =
    NULL/Return //E.
  else
    error();
```

```
match(token t) {
  if lookahead = t
    token ++ /
    get next token();
  else
    error();
}
```

Parser for

{ E → E & E
E → () along with oh
in match & error

Q1 $S \rightarrow aAB$
 $A \rightarrow aA|e$
 $B \rightarrow bB|b$

Implement recursive descent parser using this CFG & parse the string "aabb" using the resultant parser.

Q Disadvantages of recursive descent parser.

* Predictive Parse / Table Driven Parser / LL(1)

H

Final/Drive

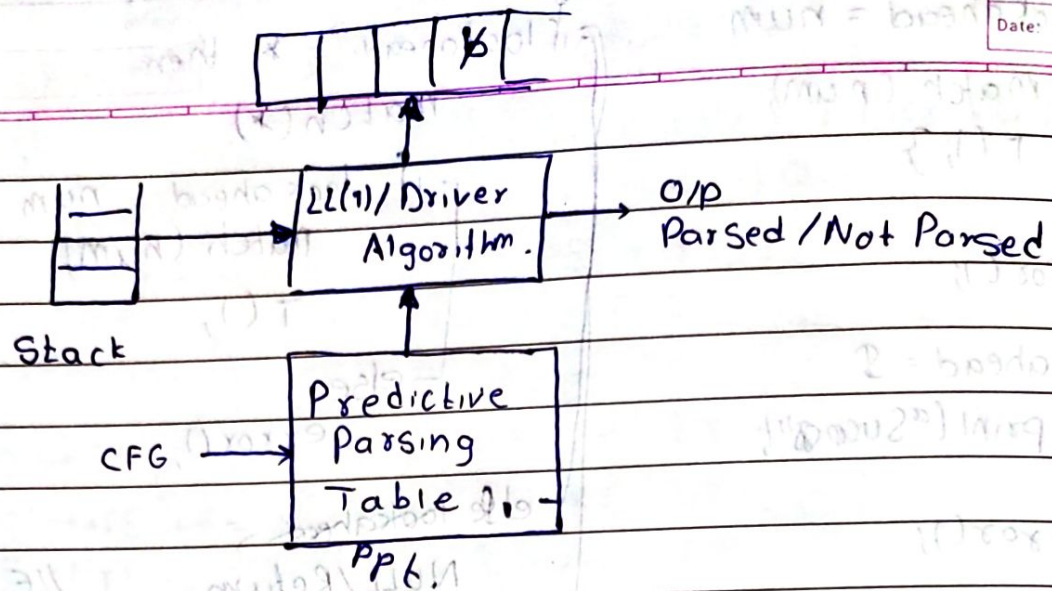


Table created by rows, columns and terminals is called predictive parsing table (PPT).

- 1) FIRST ()
 - 2) FOLLOW ()
- { Terminals }

$$\text{First}(\epsilon) = \{\epsilon\}$$

$$\text{first}(a) = \{a\}$$

$$\text{First}(A)$$

$\rightarrow$ sequential form.
 $A \rightarrow \alpha \beta$

Note: Epsilon never present in follow set

Q Calculate first & follow for given set of grammar.

$$S \rightarrow A$$

$$A \rightarrow aB \mid Ad$$

$$B \rightarrow b$$

$$C \rightarrow c$$

Variables:

$$\begin{aligned} \text{First}(S) &= \text{First}(A) = \text{First}(aB) \cup \text{First}(Ad) \\ &= \{a\} \end{aligned}$$

Follow Set of any start Symbol is \$ always

Variable	First	Follow
S	{a}	{\$}
A	{a}	{\$, d}
B	{b}	{\$, d}
C	{g}	ϕ

$A \rightarrow aBe$
 $\leftarrow C$ not present at right
 $\rightarrow AE$

So, C is useless Symbol. It never comes in process of derivation.

ex. $A \rightarrow \alpha B \beta$

$\text{Follow}(B) = \beta$

$A \rightarrow \alpha \beta$

$\text{Follow}(B) = \text{Follow}(A)$

ex. $S \rightarrow A a B D h$
 $B \rightarrow c C$
 $C \rightarrow b C \mid \epsilon$
 $D \rightarrow E F$
 $E \rightarrow g \mid \epsilon$
 $F \rightarrow G \mid \epsilon$

Variable	First	Follow
S	{a}	{\$}
B	{c}	{g, f, h}
C	{b, ϵ }	{g, f, h}
D	{g, ϵ , f}	{h}
E	{g, ϵ }	{f, h}
F	{f, ϵ }	{h}

$F \in F$
 $F(E) = F(D)$

$\text{Follow}(B) = \text{Follow}(Dh)$

We cannot add ϵ in Follow Set we just add its effect
 ex $\text{Follow}(B) = \text{First}(Dh)$
 $= \{g, f, h\}$

$E \rightarrow TE'$
 $E' \rightarrow +TE' | \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' | \epsilon$
 $F \rightarrow (E) | id.$

Variable	First(Var)	Follow(Var)
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ \$,) }
T'	{ *, ϵ }	{ \$,) }
F	{ (, id }	{ \$ }

For the production rule

$X \rightarrow Y_1 Y_2 Y_3$
 $Y_1 \rightarrow P | \epsilon$
 $Y_2 \rightarrow Q$
 $Y_3 \rightarrow R$

then $First(X) = First(Y_1, Y_2, Y_3) = \{P, Q\}$

if $First(Y_1)$ contain ϵ

then $First(X) = First(Y_1) \cup First(Y_2)$

if $First(Y_2)$ contain ϵ

$First(X) = First(Y_1) \cup First(Y_2) \cup First(Y_3)$

LP.
Lookahead
Shift
SP.

$E \rightarrow TE'$
 $E' \rightarrow +TE' | \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' | \epsilon$
 $F \rightarrow (E) | id.$

$M[E, id]$ Production
 when ϵ of stack is E
 id what is Product?

	id	+	*	(	)	\$
E	$E \rightarrow TE'$	-	-	$E \rightarrow TE'$	-	-
E'	-	$E' \rightarrow +TE'$	-	-	$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	-	-	$T \rightarrow FT'$	-	-
T'	$T' \rightarrow *FT'$	-	$T' \rightarrow *FT'$	-	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$	-	-	$F \rightarrow (E)$	-	-

Use first always, Only use follow when ϵ is occurred
 when SP pointing to terminal we perform match. ($SP == LP$)

Stack	Input	Action (PPT)
\$	id + id * id \$	Push start Symbol
\$E	id + id * id \$	$E \rightarrow TE'$
\$E'T	id + id + id \$	$T \rightarrow FT'$
\$E'T'F	id + id * id \$	$F \rightarrow id$
\$E'T'id	id + id * id \$	Pop / match
\$E'T'	+ id * id \$	$T' \rightarrow \epsilon$
\$E'	+ id * id \$	$E' \rightarrow TE'$
\$E'T'	+ id * id \$	Pop / Match
\$E'T	id * id \$	$T \rightarrow FT'$
\$E'T'F	id * id \$	$F \rightarrow id$
\$E'T'id	id * id \$	Pop / Match
\$E'T'	* id \$	$T' \rightarrow *E'T'$
\$E'T'F*	* id \$	Pop / Match
\$E'T'F	* id \$	$F \rightarrow id$
\$E'T'id	id \$	Pop / Match
\$E'T'	\$	$T' \rightarrow \epsilon$
\$E'	\$	$E' \rightarrow \epsilon$
\$	\$	Pop / Match
		Accepted

If not match
 Unsuccessful
 Parsing

When Stack & IP tapes are empty at same time
 the string is accepted

Construct ppt for the following Set of grammar.

$S \rightarrow aA / bAc$

$A \rightarrow c | \epsilon$

Variable	First	Follow
S	{a, b}	{ \$ }
A	{ c, ε }	{ g, c }

	a	b	c	\$
S	$S \rightarrow aA$	$S \rightarrow bAc$	-	-
A			$A \rightarrow c$ $A \rightarrow \epsilon$	$A \rightarrow \epsilon$

It is not LL(1) grammar.
as we get two entries in
table for $M[A, c]$.

Check if given gram. is LL(1)

- Left recursion } Any of 3
 2) Ambiguous }
 3) PPT }

Parse the input string "acgh" by using predictive parsing algorithm with the help of given grammar

$S \rightarrow aBDh$
 $B \rightarrow cC$

	Variable	First	Follow
$C \rightarrow bC \epsilon$	S	{ a }	{ \$ }
$D \rightarrow EF$	B	{ c }	{ g, f, h }
$E \rightarrow g \epsilon$	C	{ b, ε }	{ g, f, h }
$F \rightarrow f \epsilon$	D	{ g, f, ε }	{ h }
	E	{ g, ε }	{ f, h }
	F	{ f, ε }	{ h }

OK

	a	b	c	f	g	h	\$
S	$S \rightarrow aBDh$	-	-	-	-	-	-
B	-	-	$B \rightarrow cC$	-	-	-	-
C	-	$C \rightarrow bC$	-	$C \rightarrow e$	$C \rightarrow e$	$C \rightarrow g$	-
D	-	-	-	$D \rightarrow EF$	$D \rightarrow EF$	$D \rightarrow EF$	-
E	-	-	-	$E \rightarrow S$	$E \rightarrow g$	$E \rightarrow g$	-
F	-	-	-	$F \rightarrow f$	-	$F \rightarrow g$	-

Stack	Input	Action
\$	acgfh\$	push Start Symbol
\$S	acgfh\$	$S \rightarrow aBDh$
\$hDBa	acgfh\$	Pop
\$hDB	cghf\$	$B \rightarrow cC$
\$hDCc	cghf\$	Pop
\$hDC	ghf\$	$C \rightarrow e$
\$hD	ghf\$	$D \rightarrow EF$
\$hFE	ghf\$	$E \rightarrow g$
\$hFg	ghf\$	Pop
\$hF	fh\$	$F \rightarrow f$
\$hf	fh\$	Pop
\$h	h\$	Pop
\$	\$	Accept.

* Bottom Up parsing

Shift Reduce

LR

LR(0)

SLR(1)

CLR(1)

LALR(1)

$$E \rightarrow E + E \mid E * E \mid (E) \mid id.$$

Page No.:

Date:

YOUVA

$$\Rightarrow \underline{id} + id * id$$

underlined part is called handle.

$$= E + \underline{id * id}$$

$$= E + E * id$$

$$= \underline{E * id}$$

Tree pruning

$$= \underline{E * E}$$

$$= \underline{E}$$

possible handles = $\{ id, E + E, E * E \}$

• Handle is a sentential form present on the right hand side such that a substring β that matches the RHS of a production $A \rightarrow \beta$

• The reduction of β to A is a state along the reverse of rightmost derivation.

1 Shift Action: Parser shift the next token onto the stack.

2 Reduce Action: Parser reduces the RHS of a production to its LHS.

3 error: Parser discovered a syntax error.

4. Accept Action accept describe a successful parser.

Stack	Input	Action
\$	id + id * id \$	Shift
\$ id	+ id * id \$	Reduce $E \rightarrow id$
\$ E	+ id * id \$	Shift
\$ E +	id * id \$	Shift
\$ E + id	* id \$	Reduce $E \rightarrow id$
\$ E + E	* id \$	Reduce $E \rightarrow E + E$
\$ E	* id \$	Shift
\$ E *	id \$	Shift
\$ E * id	\$	Reduce $E \rightarrow id$

\$ E * E

\$

reduce $E \rightarrow E * E$

M T W T F S S						
Page No.:				YOUVA		
Date:						

\$ E

\$

Accept

When input is \$ and stack has start symbol we accept.

Q How many shift & reduce actions are req to parse the input string $id * (id + id)$, using $E \rightarrow E + E \mid E * E \mid (E) \mid id$.

Stack	Input	Action
\$	$id * (id + id) \$$	Shift
$\$ id$	$* (id + id) \$$	reduce $E \rightarrow id$.
$\$ E$	$* (id + id) \$$	Shift
$\$ E *$	$(id + id) \$$	Shift
$\$ E * ($	$id + id) \$$	Shift
$\$ E * (id$	$+ id) \$$	reduce $E \rightarrow id$.
$\$ E * (E$	$+ id) \$$	Shift
$\$ E * (E +$	$id) \$$	Shift
$\$ E * (E + id$	$) \$$	reduce $E \rightarrow id$
$\$ E * (E + E$	$) \$$	reduce reduce $E \rightarrow E + E$
$\$ E * (E$	$) \$$	Shift
$\$ E * (E)$	$\$$	Reduce $E \rightarrow (E)$
$\$ E * E$	$\$$	Reduce $E \rightarrow E * E$
$\$ E$	$\$$	Accept

$id * (id + id)$

↓

(terminal)

reduce

↓

Count

Underline

$S \rightarrow (L) \mid a$

$(a; (a, a))$

$L \rightarrow L, S \mid S$

$(S, (a, a))$

a-reduce

$(a, (a, b))$

$(L, (a, a))$

a-shift

$(L, (S, a))$

to parse this tree

$(L, (L, a))$

$(L, (L, S))$

$(L, (L))$

(L, S)

(L)

S